AI and Full Stack Development Internship Report **Submitted by:** AUMNAMAHA **Department:** Computer Science and Engineering **Institution:** GITAM Deemed to be University, Hyderabad Campus **Internship Duration:** 23rd May 2026 â€“ 23rd June 2026 **Organization:** Swecha Free Software Foundation --- ## ðŸ“‹ Table of Contents 1. [Declaration](#declaration) 2. [Certificate](#certificate) 3. [Acknowledgement](#acknowledgement) 4. [Abstract](#abstract) 5. [Chapter 1: Introduction to Artificial Intelligence](#chapter-1-introduction-to-artificial-intelligence) 6. [Chapter 2: Full Stack Development](#chapter-2-full-stack-development) 7. [Chapter 3: Modern Software Development Practices](#chapter-3-modern-software-development-practices) 8. [Chapter 4: Project Implementation](#chapter-4-project-implementation) - [Project 1: Vaani â€” On-Device AI Voice Assistant](#project-1-vaani---on-device-ai-voice-assistant) - [Project 2: SAHAY â€” Government Scheme RAG Chatbot](#project-2-sahay---government-scheme-rag-chatbot) - [Project 3: AI Automobile Advisor](#project-3-ai-automobile-advisor) 9. [Chapter 5: Internship Timeline & Learning Outcomes](#chapter-5-internship-timeline--learning-outcomes) 10. [Conclusion](#conclusion) --- ## Declaration I hereby declare that this internship report entitled "**AI and Full Stack Development Internship at Swecha Free Software Foundation**" is an original work carried out by me during my one-month summer internship (May 23, 2026 â€“ June 23, 2026). The work presented in this report represents my learning, contributions, and experiences during the internship period. This report has been prepared as part of the requirements for the completion of Bachelor of Technology degree in Computer Science and Engineering at GITAM Deemed to be University. **Name:** AUMNAMAHA **Roll Number:** 2023XXXXXX **Date:** _______________ --- ## Certificate This is to certify that **AUMNAMAHA** has undergone the AI and Full Stack Development Internship program at Swecha Free Software Foundation from May 23, 2026 to June 23, 2026. The intern gained practical experience in: - Artificial Intelligence-assisted development - On-device machine learning model deployment - React Native mobile app development - Full-stack Streamlit application development - RAG (Retrieval-Augmented Generation) systems - Privacy-first architecture design - Specification-driven development - GitLab version control and CI/CD practices The internship was completed under the guidance of Swecha Free Software Foundation, where the intern gained practical exposure to real-world software engineering workflows and AI-powered development methodologies. **Guide:** _______________________ **Signature:** ___________________ **Head of Department:** _______________________ **Signature:** ______________________ --- ## Acknowledgement I would like to express my sincere gratitude to **Swecha Free Software Foundation** for providing me with the opportunity to complete this summer internship as an AI and Full Stack Development Intern. I am grateful to my mentors and team members at Swecha for their valuable guidance, continuous support, and encouragement throughout the internship period. Their practical insights into software development methodologies, artificial intelligence tools, on-device ML deployment, and open-source development practices helped me improve my

technical and professional skills. I am thankful to the GITAM faculty members and department for providing the necessary support and guidance required for successfully completing this internship. During this internship, I gained practical exposure to: - **On-device AI:** Whisper transcription and Qwen SLM inference using native bindings - **Privacy-first architecture:** Zero cloud dependencies with local SQLite storage - **React Native/Expo:** Building cross-platform mobile applications - **Streamlit RAG systems:** Multi-language government scheme finder - **Full-stack development:** React frontend, Python backend, and database integration - **Specification-driven development:** Using .specify folder for code documentation I would also like to thank my teammates and everyone who contributed to making this internship a valuable learning experience. --- ## Abstract The summer internship at Swecha Free Software Foundation provided practical exposure to modern software development practices combining Artificial Intelligence, Mobile Development (React Native), Full-Stack Web Applications (Streamlit), and privacy-first architecture. During the one-month internship, I learned and applied concepts such as: - **AI-assisted software development:** Using LLMs for code generation, documentation, and debugging - **On-device machine learning:** Deploying Whisper and Qwen models locally on mobile devices - **Privacy-first architecture:** Building air-gapped applications with zero cloud dependencies - **Specification-driven development:** Writing clear technical specifications before implementation - **React Native/Expo:** Cross-platform mobile app development for iOS and Android - **RAG systems:** Retrieval-Augmented Generation for semantic search in government schemes - **Multi-language UIs:** Supporting English, Hindi, and Telugu interfaces ### Projects Completed: **Project 1: Vaani** â€” A privacy-first, offline voice-to-structured-data extraction mobile app that uses on-device Whisper for speech transcription and Qwen SLM for schema extraction, storing all data in local SQLite without any cloud dependencies. **Project 2: SAHAY (Government Scheme RAG Chatbot)** â€” A trilingual Streamlit application (English/Hindi/Telugu) that helps users find government schemes using Gemini API with PDF document support and knowledge base retrieval. **Project 3: AI Automobile Advisor** â€” A Streamlit-powered automotive advisory chatbot for vehicle-related queries, documentation guidance, and model recommendations. This internship significantly enhanced my understanding of real-world software engineering practices, AI-powered development workflows, mobile application development, full-stack web development, and privacy-first architecture design. --- # Chapter 1: Introduction to Artificial Intelligence ## 1.1 What is Artificial Intelligence? Artificial Intelligence (AI) is a branch of computer science that focuses on creating systems capable of performing tasks that normally require human intelligence, such as learning, reasoning, decision-making, and problem-solving. Modern AI encompasses various subfields including: - **Machine Learning:** Systems that learn patterns from data - **Natural Language Processing (NLP):** Understanding and generating human language - **Computer Vision:** Interpreting and analyzing visual information - **Generative AI:** Creating new content (text, images, code) ## 1.2 Applications of Artificial Intelligence During this internship, we leveraged AI in several practical applications: | Application Area | Use Case | Technology Used | |-----------------|----------|----------------| | Speech-to-Text | Voice note transcription | Whisper Tiny EN (on-device) | | Text Classification | Parsing unstructured notes into structured data | Qwen2.5-0.5B-Instruct | | RAG Chatbots | Government scheme queries | Gemini 2.5 Flash + Vector retrieval | | Code Generation | Documentation and helper functions | GitHub Copilot / Cursor | | API Assistants | Natural language to API calls | Function calling patterns | ## 1.3 Generative AI and Large Language Models (LLMs) ### What are LLMs? Large Language Models are neural network models trained on vast amounts of text data that can understand and generate human-like text. Examples include: - **Whisper:** OpenAI's speech recognition model - **Qwen:** Alibaba's large language model series - **Gemini:** Google's multimodal AI model - **Llama:** Meta's open-source LLM family ### Small Language Models (SLMs) For on-device deployment, smaller quantized models are preferred: - **Qwen2.5-0.5B-Instruct:** 0.5 billion parameters, runs efficiently on mobile CPUs - Quantized using GGUF format for native inference engines ### On-Device AI Benefits: ``` â”Œâ”€€ â”, PRIVACY-FIRST ON-DEVICE AI â”, â”œâ”€â â”, â”, â”, âœ“ Zero cloud dependencies â”, â”, âœ“ No data leaves the device â”, â”, âœ“ Works offline â”, â”, âœ“ No API keys required â”, â”, âœ“ No external network calls â”, â”, âœ“ HIPAA/GDPR compliant â”, â”, â”, â””â”€â ``` ## 1.4 AI in Software Development ### AI-Assisted Coding AI-assisted coding involves using LLMs to: - Generate boilerplate code - Explain complex functions - Suggest improvements and refactoring - Write documentation - Debug errors - Convert code between languages ### Prompt Engineering Fundamentals Effective prompt engineering includes: 1. **Providing Context:** Give the model background information 2. **Specifying Format:** Clearly state expected output format 3. **Few-Shot Examples:** Include examples of desired behavior 4. **Iterative Refinement:** Improve prompts based on outputs 5. **Chain-of-Thought:** Guide the model through reasoning steps **Example Prompt for Schema Extraction:** ``` You are a structured data extraction assistant. Parse the following transcript into the specified JSON schema: Schema: { "type": "object", "properties": { "title": {"type": "string"}, "category": {"type": "string", "enum": ["expense", "task", "reminder"]}, "amount": {"type": "number"} }, "required": ["title", "category"] } Transcript: "I spent money on groceries today" Output JSON only. ``` --- # Chapter 2: Full Stack Development ## 2.1 Introduction to Full Stack Development Full Stack Development refers to building complete web applications by working on both frontend (user interface) and backend (server logic, database) components. ### Application Architecture Layers: ``` â”Œâ”€€ â”, Full-Stack App

â”‚ â”œâ”€â
â”‚ â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”�
â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”�
â”‚ â”‚ â”‚ Frontend UI â”‚ â”‚ Backend API â”‚ â”‚ â”‚ â”‚ (React Native) â”‚â—
„â”€â”€â–ºâ”‚ (Python/Node)
â”‚ â”‚ â”‚ â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜
â””â”€â”€â”€â”€â”€â”€â”¬â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜
â”‚ â”‚ â”‚ â”‚ â”‚ â–¼ â”‚ â”‚ â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”�
â”‚ â”‚ â”‚ Database â”‚ â”‚ â”‚ â”‚ (SQLite)
â”‚ â”‚ â”‚ â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜
â”‚ â””â”€â
``` ## 2.2 Frontend Development Technologies ### React Native & Expo SDK 51 React Native allows building native mobile apps using React components: | Package | Purpose | Version | |---------|---------|---------| | `expo` | Mobile development framework | ~51.0.0 | | `react-native` | Cross-platform UI components | 0.74.5 | | `expo-router` | File-based routing | Latest | | `expo-file-system` | Local file operations | ~17.0.1 | | `expo-av` | Audio/video capture | ~14.0.7 | ### Native Modules for ML Inference Vaani uses native bindings for on-device ML: ```bash # Whisper for speech-to-text whisper.rn # whisper.cpp React Native binding # Llama for text generation llama.rn # llama.cpp React Native binding ``` **Note:** These native modules require building a custom Expo app â€” they cannot run in Expo Go. ## 2.3 Backend Development with Streamlit Streamlit enables rapid prototyping of data apps with minimal code: | Feature | Description | |---------|-------------| | **Declarative UI** | Build interfaces by defining state, not imperative commands | | **Auto-Updating** | Python script re-runs when data changes | | **Built-in Charts** | Plotly-based visualization support | | **File Uploads** | Drag-and-drop document support | ### Key Streamlit Components Used: ```python import streamlit as st # Page configuration st.set_page_config(page_title="SAHAY â€” Government Scheme Finder", layout="wide", initial_sidebar_state="expanded") # Columns for metrics display col1, col2, col3, col4 = st.columns(4) # Tabs for multi-section UI tab1, tab2, tab3 = st.tabs(["Chat", "Schemes", "How to Apply"]) ``` ## 2.4 Database Management ### SQLite â€” Local Relational Storage SQLite is used for local persistence without external servers: ```python import sqlite3 # Initialize database conn = sqlite3.connect('vaani.db') cursor = conn.cursor() # Create tables with indexes cursor.execute(''' CREATE TABLE IF NOT EXISTS records (id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT NOT NULL, category TEXT NOT NULL, content TEXT, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP) ''') # Index for fast searching cursor.execute('CREATE INDEX IF NOT EXISTS idx_category ON records(category)') ``` ### SQLAlchemy ORM Pattern For complex applications, SQLAlchemy provides abstraction: ```python from sqlalchemy import create_engine, Column, Integer, String from sqlalchemy.orm import declarative_base Base = declarative_base() class Record(Base): __tablename__ = 'records' id = Column(Integer, primary_key=True) title = Column(String(200),
```

nullable=False) category = Column(String(50), nullable=False) def __repr__(self): return f"" ``` --- # Chapter 3: Modern Software Development Practices ## 3.1 Markdown Documentation Markdown is used for project documentation, README files, and specification writing: ### Key Documentation Files: - **README.md:** Quick start guide, installation instructions - **implementation.md:** Detailed technical implementation notes - **DEMO.md:** Feature walkthrough with screenshots - **.specify/**: Code specifications before implementation ### Specification-Driven Development Before implementing code, write specifications in `.specify/`: ```markdown # .specify/voice-recorder.md ## Component: VoiceRecorder ### Responsibilities 1. Capture audio via expo-av microphone API 2. Store WAV files in app directory 3. Notify user of recording state changes ### Props - `onRecordingStart()`: Callback when mic opens - `onRecordingStop()`: Callback with audio file path ### State - `isRecording: boolean` - `audioPath: string | null` ### Events - `onError(error): void` ``` ## 3.2 Version Control with GitLab GitLab is used for: - Code versioning and branching - Issue tracking and merge requests - CI/CD pipeline configuration ### Example .gitlab-ci.yml Configuration: ```yaml stages: - lint - test - build lint: stage: lint script: - npm run typecheck - npm run eslint test: stage: test script: - npm run test:coverage artifacts: paths: - coverage/ build: stage: build script: - npx expo export --platform web artifacts: paths: - dist/ ``` ## 3.3 Code Quality Practices ### Type Safety with TypeScript Vaani uses TypeScript for type safety: ```typescript // types/record.ts export interface Expense { title: string; amount: number; currency: 'INR' | 'USD' | 'EUR'; category: string; date: Date; } export type RecordType = 'expense' | 'task' | 'reminder'; export interface DatabaseRecord { id: number; title: string; category: RecordType; content?: string; createdAt: Date; } ``` ### ESLint Configuration: ```jsonc // .eslintrc.json { "root": true, "extends": ["expo", "plugin:@typescript-eslint/recommended"], "parser": "@typescript-eslint/parser", "rules": { "@typescript-eslint/no-unused-vars": "warn", "no-console": "off" } } ``` --- # Chapter 4: Project Implementation **Project Summary:** During the four-week internship period (May 23 â€“ June 23, 2026), I completed three distinct software development projects covering mobile app development, full-stack web development, and AI integration. The weekly activity log below outlines the progressive learning and implementation journey. ### Weekly Activity Summary Table: | Week | Focus Area | Projects Advanced | Key Deliverables | |------|------------|-------------------|------------------| | **Week 1**
(May 23â€“29) | Environment Setup & AI Fundamentals | â€” | âœ“ GitLab setup
âœ“ AI/ML concepts reviewed
âœ“ Expo SDK 51 study complete
âœ“ Native module patterns learned | | **Week 2**
(May 30â€“June 5) | Vaani Core Development | Vaani v1.0 | âœ“ On-device Whisper integration
âœ“ Qwen SLM schema extraction
âœ“ SQLite database implementation
âœ“ Dashboard UI complete

âœ“ Benchmarking dashboard added | | **Week 3**
(June 6â€“12) | SAHAY RAG Development | SAHAY v1.0 | âœ“ Multi-language UI (EN/HI/TL)
âœ“ Profile filtering logic
âœ“ Gemini API RAG integration
âœ“ PDF document parsing
âœ“ Scheme database populated | | **Week 4**
(June 13â€“23) | Automobile Advisor & Wrap-up | Auto v1.0, Report | âœ“ Starlette backend setup
âœ“ Chat interface with history
âœ“ Streamlit Cloud deployment
âœ“ Documentation completed
âœ“ Final report written | ## Project 1: Vaani â€” On-Device AI Voice Assistant ### 4.1 Overview **Vaani** (Sanskrit for "voice") is a privacy-first, fully offline mobile application that converts spoken notes into structured data without sending any data to the cloud. It runs entirely on-device using Whisper for speech transcription and Qwen SLM for schema extraction. **Use Cases:** - Recording expenses: "I spent â‚¹500 at Starbuck" â†’ {title: "Starbucks", amount: 500, currency: "INR"} - Capturing tasks: "Call mom to schedule dentist appointment" â†’ {title: "Schedule dentist", category: "task"} - Setting reminders: "Take medicine at 8pm" â†’ {title: "Take medicine", type: "reminder", due_date: "2026-07-29T20:00:00Z"} ### 4.2 Tech Stack | Category | Technology | Purpose | |----------|-----------|---------| | **Mobile Framework** | Expo SDK 51 | Cross-platform development | | **Language** | TypeScript / React Native | Type-safe UI components | | **Speech-to-Text** | whisper.rn (Whisper Tiny EN) | On-device transcription | | **Schema Extraction** | llama.rn (Qwen2.5-0.5B) | Parse text â†’ structured JSON | | **Storage** | expo-sqlite | Local SQLite database | | **File Operations** | expo-file-system | Model downloads, report export | | **Network** | @react-native-community/netinfo | WiFi check before downloads | ### 4.3 System Architecture ```ascii

â”Œâ”€
â”‚ Vaani Application
â”‚ â”œâ”€
â”‚ â”‚ â”‚ â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”�
â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”�
â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”�
â”‚ â”‚ â”‚ Voice â”‚ â”‚ Transcription â”‚ â”‚ Schema â”‚ â”‚ â”‚ â”‚ Recorder
â”‚â”€â”€â”€>â”‚ Engine â”‚â”€â”€â”€â”€>â”‚ Extraction â”‚ â”‚ â”‚ â”‚ â”‚ (expo-av)
â”‚ â”‚ (whisper.rn) â”‚ â”‚ (llama.rn/Qwen)
â”‚ â”‚ â”‚ â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜
â””â”€â”€â”€â”€â”€â”€â”€â”€â”¬â”€â”€â”€â”€â”€â”€â”€â”€â”˜
â””â”€â”€â”€â”€â”€â”€â”€â”€â”¬â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜
â”‚ â”‚ â”‚ â”‚ â”‚ â”‚ â–¼ â–¼
â”‚ â”‚ â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€

â”‚ â”‚ â”‚ Benchmark Metrics Logger â”‚ â”‚ â”‚ â”‚ â€¢ Transcription latency (ms)
â”‚ â”‚ â”‚ â”‚ â€¢ Extraction latency (ms) â”‚ â”‚ â”‚ â”‚ â€¢ CPU usage (%)
â”‚ â”‚ â”‚ â”‚ â€¢ RAM usage (MB)
â”‚ â”‚ â”‚ â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”¬â”€â”€â”€â”€â”€â”€â”€
â”‚ â”‚ â”‚ â”‚ â”‚ â–¼
â”‚ â”‚ â”Œâ”€
â”‚ â”‚ â”‚ SQLite Database
â”‚ â”‚ â”‚ â”‚ â”‚ â”‚ â”‚ â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”�
â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”� â”‚ â”‚ â”‚ â”‚ expenses
â”‚ â”‚ benchmarks â”‚ â”‚ â”‚ â”‚ â”œâ”€â”€ id (PK)
â”‚ â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜ â”‚ â”‚ â”‚ â”œâ”€â”€ title
â”‚ â”‚ â”‚ â”‚ â”œâ”€â”€ category â”‚ â”‚ â”‚ â”‚ â”œâ”€â”€ content
â”‚ â”‚ â”‚ â”‚ â””â”€â”€created
â”‚ â”‚ â”‚ â”‚ â””â”€â”˜
â”‚ â”‚ â””â”€
â”‚ â”‚ â”‚ â””â”€
Data Flow: 1. User speaks into microphone â†’ WAV file recorded (16kHz PCM) 2. whisper.rn transcribes to text (on-device GPU/CPU) 3. Qwen2.5-0.5B extracts schema fields into JSON 4. Results stored in SQLite with index for fast search ``` ### 4.4 File Structure ``` Vaani/ â”œâ”€â”€ app/ # React Native components â”‚ â”œâ”€â”€ (tabs)/ # Tab-based navigation â”‚ â”‚ â”œâ”€â”€ onboarding/ # Name entry screen â”‚ â”‚ â”œâ”€â”€ home/ # Main dashboard â”‚ â”‚ â””â”€â”€ review/ # Transcript â†’ structured data view â”‚ â”œâ”€â”€ _layout.tsx # Root layout with navigation â”‚ â””â”€â”€ index.tsx # Home screen â”œâ”€â”€ constants/ # App-wide constants â”œâ”€â”€ db/ # Database schema definitions â”œâ”€â”€ services/ # ML inference wrappers â”‚ â”œâ”€â”€ whisperService.ts # Speech transcription logic â”‚ â””â”€â”€ llamaService.ts # Schema extraction logic â”œâ”€â”€ specs/ # Code specifications â”‚ â””â”€â”€ .specify/ # Pre-implementation docs â”œâ”€â”€ types/ # TypeScript type definitions â”œâ”€â”€ utils/ # Helper functions â”œâ”€â”€ android/ # Android native project â”œâ”€â”€ ios/ # iOS native project â”œâ”€â”€ package.json # Dependencies and scripts â””â”€â”€ README.md # Documentation ``` ### 4.5 Key Code Snippets #### Speech-to-Text Transcription Function ```typescript // services/whisperService.ts import whisper, { WhisperModel, InitOptions } from "whisper.rn"; let model: WhisperModel | null = null; export async function initWhisper(onProgress: (progress: number) => void): Promise { // Initialize Whisper model with streaming callback model = await whisper({ modelPath: getAsset("whisper-tiny-en.bin"), onProgress: onProgress, device: "gpu" as const, // Use GPU if available dtype: "f16" as const, // Float16 for faster inference }); } export async function transcribeAudio(audioPath: string, onError: (error: Error) => void): Promise { if (!model) throw new Error("Whisper not initialized"); const result = await model.transcribe({ filePath: audioPath, stream: true, // Stream for real-time feedback }); return result.text; } ``` #### Schema Extraction with Qwen SLM ```typescript // services/llamaService.ts import llama from "llama.rn"; let

qwenModel: any = null; export async function loadQwen(onProgress: (progress: number) => void): Promise { const modelPath = getAsset("qwen2.5-0.5b-instruct-q4_k_m.gguf"); qwenModel = await llama({ modelPath, n_ctx: 2048, // Context window size n_threads: Math.max(4, cpuCount - 1), onProgress: onProgress, }); } export async function extractSchema(transcript: string, schemaPrompt: string): Promise { if (!qwenModel) throw new Error("Qwen not loaded"); const response = await qwenModel.generate({ prompt: `Extract structured data from: "${transcript}" ${schemaPrompt}`, stop: ["```", "\n\n"], // Stop at code block markers max_tokens: 512, }); return JSON.parse(response.trim()); } // Corrective retry logic for malformed JSON export async function extractSchemaWithRetry(transcript: string, maxRetries: number = 2): Promise { let response: any; let attempt = 0; while (attempt < maxRetries) { try { response = await extractSchema(transcript, DEFAULT_PROMPT); // Validate schema fields const validFields = ["title", "category", "amount"]; if (!validFields.every(f => response[f])) { throw new Error("Incomplete schema"); } return response; } catch (error) { attempt++; if (attempt >= maxRetries) throw error; // Retry with corrected prompt const correctionPrompt = `The previous output was malformed. Extract again with the same transcript.`; response = await extractSchema(transcript, correctionPrompt); } } } ``` #### SQLite Storage Layer ```typescript // db/records.ts import { openDB } from "expo-sqlite"; let db: any = null; export async function initDatabase(): Promise { const path = getApplicationDocumentsPath() + "/vaani.db"; db = await openDB(path, 1, { upgrade(db, oldVersion, newVersion) { if (oldVersion < 1) { // Create initial schema db.exec(` CREATE TABLE records ( id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT NOT NULL, category TEXT NOT NULL, content TEXT, created_at DATETIME DEFAULT CURRENT_TIMESTAMP ) `); } }, }); } export async function saveRecord(record: any): Promise { if (!db) throw new Error("Database not initialized"); await db.run(` INSERT INTO records (title, category, content) VALUES (:title, :category, :content)`, { title: record.title, category: record.category, content: record.content }); } export async function searchRecords(category: string | null = null, titleContains: string | null = null): Promise { if (!db) throw new Error("Database not initialized"); let query = "SELECT * FROM records WHERE 1=1"; const params: any = {}; if (category) { query += " AND category = :category"; params.category = category; } if (titleContains) { query += " AND title LIKE :title"; params.title = `%${titleContains}%`; } return db.all(query, params); } ``` ### 4.6 Main App Flow (from app.py / app.json) **Onboarding Screen:** ```typescript // app/(tabs)/onboarding.tsx import { useState } from "react"; import { View, TextInput, StyleSheet } from "react-

```
native"; export default function OnboardingScreen() { const [userName,
```

saveUser(userName)} />

);
}

Home Dashboard:
```typescript
// app/(tabs)/home.tsx
export default function HomeScreen({ records }: { records
return (

{/* Expenses Section */}

{records.filter(r => r.category === "expense").map

))}

{/* Tasks Section */}

{records.filter(r => r.category === "task").map(r =>

))}

{/* Reminders Section */}

{records.filter(r => r.category === "reminder").ma

))}

);
}
```

### 4.7 Screenshot Gallery

![Vaani Onboarding](../Pictures/Vaani Pictures/Screenshot
*Figure 4.1: Vaani onboarding screen â€” enter your name

![Vaani Home Dashboard](../Pictures/Vaani Pictures/Screen
*Figure 4.2: Home dashboard showing Expenses, Tasks, ar

![Vaani Review Screen](../Pictures/Vaani Pictures/Screensh
*Figure 4.3: Review screen â€” raw transcript on left, struc

### 4.8 Learning Outcomes

From Vaani development